

wife-system · 阶段 0 ~ 3 实现后教材

P1

财务数据底座

固定代码基线 6e89762
中文学习版 · 虚拟数据示例

从实际代码、数据流和测试证据出发；每册可独立阅读。

目录

第二册：P1 财务数据底座	3
学习目标与前置知识	3
1. 为什么财务事实必须进入关系数据库	4
小练习	4
2. 分层职责：DTO、ORM、service、repository 与 migration	5
2.1 Pydantic DTO	5
2.2 SQLAlchemy ORM	5
2.3 service 与 repository	5
2.4 Alembic migration	5
小练习	5
3. 金额、账户、活动、收入与预算的关系	6
3.1 整数分与币种	6
3.2 核心关系	6
小练习	6
4. 事务、幂等、乐观锁与数据库约束	7
4.1 一个写命令的一生	7
4.2 乐观版本与行锁	7
4.3 数据库约束是最后防线	7
小练习	7
5. SQLite 与 PostgreSQL：开发速度和生产语义分工	8
缺陷带来的四个教训	8
常见误区	9
阶段练习	10
检查题	11

第二册：P1 财务数据底座

代码基线：**6e89762**。P3 后来扩展了活动导入字段；本册会明确区分 P1 基础与后续扩展。

学习目标与前置知识

你将理解为什么聊天记录不能替代账本，SQLAlchemy、Pydantic、service、repository、session 和 Alembic 分别解决什么问题，以及事务、幂等、版本和数据库约束怎样共同保护财务事实。

前置知识：Python 类与异常、SQL 的表/行/主键概念。未系统学过数据库也可以按数据流学习。

首次术语：

- **ORM**（对象关系映射）：把 Python 类映射为关系数据库表，并用对象表达行。
- **migration（迁移）**：版本化的数据库结构变更程序，使旧库按明确顺序升级到新结构。
- **事务**：一组数据库操作作为一个整体提交；中途失败时全部回滚。
- **session**：SQLAlchemy 的工作单元，跟踪对象、执行查询并划定事务范围。

1. 为什么财务事实必须进入关系数据库

聊天记忆适合保留上下文，不适合当账本：模型回答可能变化，消息可被截断，多个入口会并发，且聊天文本没有外键、唯一约束、原子提交和可重复查询。

关系数据库把事实拆成有约束的表：账户有稳定 ID；交易头说明一次业务事件；分录说明金额落在哪个账户或分类；回执证明某来源命令是否已处理；审计记录变更事实。数据库能回答“截至某时刻余额是多少”，也能拒绝孤儿引用和重复来源。

本项目使用交易头 + 分录，而不是一张简单收支表。模型见 `finance/models.py` 第 100 ~ 141 行：
`FinancialTransaction` 保存种类、发生时间、关联原交易和回执；`TransactionEntry` 保存有正负方向的整数分、账户或分类角色。

虚拟支出 18 元可表示为：

```
financial_transaction(kind=expense)
  entry #1: account -1800  (账户资产减少)
  entry #2: expense +1800  (餐饮支出增加)
合计: 0
```

这样同一模型可表达：

业务	交易头	典型分录
收入	income	账户增加、收入分类相抵
支出	expense	账户减少、支出分类相抵
转账	transfer	一个账户减少、另一个增加
退款	refund	关联原支出并反向部分分录
冲销	reversal	关联原交易并完整反向

代付、报销和分期在 P1 冻结中延期，不能仅凭已有 `kind` 猜出业务规则。

小练习

用纸画一笔“虚拟钱包向虚拟银行卡转 100 元”的交易头和两条分录。检查所有分录的 `amount_minor` 之和是否为 0。

2. 分层职责：DTO、ORM、service、repository 与 migration

Pydantic 命令 DTO

- FinanceService (业务规则与事务)
- Repository (同一 session 内查询/写入)
- SQLAlchemy ORM 模型
- 数据库约束
- Alembic migration 创建真实结构

2.1 Pydantic DTO

DTO 描述公开命令的字段和类型，例如账户 ID、金额字符串、来源事件。它在数据库之前拒绝 float、缺字段和额外字段。DTO 不是数据库表，也不会自动提交。

2.2 SQLAlchemy ORM

Base 与引擎在 `finance/db.py` 第 19 ~ 42 行。`make_engine()` 为 SQLite 打开外键；`make_session_factory()` 产生短生命周期 session。模型从 `Account`、`Category`、`CommandReceipt` 一直定义到账本、活动、收入和预算。

2.3 service 与 repository

`FinanceService` 在 `finance/service.py` 第 157 ~ 158 行明确写着：同步命令服务，每次写入拥有一个短事务。`repository` 在 `finance/repositories.py` 第 11 ~ 42 行只做 session 范围的查询、锁、add 和 flush；事务所有权留在 service，避免底层函数偷偷提交。

2.4 Alembic migration

P1 首个迁移是 `bfc163b9b8e9_create_finance_foundation.py`，后续 `1377551283d0_enforce_sqlite_integer_minor_storage.py` 补齐 SQLite 金额存储类别约束。migration 才是把模型意图变成已部署数据库结构的过程；只改 ORM 不会自动安全升级已有库。

未采用方案：项目选择 SQLAlchemy 2.x 显式 ORM 模型，而不是 `SQLModel`。显式模型让复杂约束、复合外键、版本字段和 Alembic 差异更清楚；简单 CRUD 原型可考虑 `SQLModel`，但这里的账本与迁移需要更直接控制。

小练习

选择 `create_account`：分别写出 DTO、service、repository、ORM 和 migration 在这条路径上承担的一句话职责。

3. 金额、账户、活动、收入与预算的关系

3.1 整数分与币种

项目把 18 元存为 **1800** 分，不用二进制 float。金额解析与舍入见 `finance/money.py` 第 12 ~ 55 行：当前只允许 CNY，输入按 Decimal 解析，舍入采用明确规则，数据库落整数 minor units。

整数分的优点是加总精确、JSON 简单、跨 Python/PostgreSQL 一致。若系统未来必须支持三位小数币种或任意精度计息，**NUMERIC/Decimal** 会更合适；当前 CNY 账本采用整数分，显示层再格式化为元。

3.2 核心关系

```

Account —┐
           └─ TransactionEntry — FinancialTransaction — CommandReceipt
Category ┘

ActivityTemplate → immutable ActivityTemplateRevision → ActivityOccurrence
ActivityOccurrence → ActivityEntryAllocation → expense TransactionEntry

IncomeSchedule → IncomeScheduleVersion → IncomeExpectation
IncomeExpectation → IncomeExpectationMatch → income TransactionEntry

BudgetPlan → immutable BudgetVersion → BudgetAllocation → Category
  
```

代码位置：活动模板、修订在 `models.py` 第 144 ~ 178 行，活动发生和支出分配在第 259 ~ 280 行，收入安排链在第 283 ~ 334 行，预算版本链在第 337 ~ 376 行。最终文件中的 `reference_min_minor`、`reference_max_minor` 和导入候选来源是 P3 扩展，不应倒写成 P1 首版已有能力。

关键不变量：

1. 一笔交易的分录总和为 0。
2. 金额字段保存整数分，币种当前为 CNY。
3. 历史修订和预算版本不原地改写；新变化产生新版本。
4. 活动发生绑定具体模板修订，历史解释不会随模板当前版本漂移。
5. 删除策略以 **RESTRICT** 和归档为主，避免账本引用被物理删除。

小练习

说明“周末采购模板改名”为什么不能回写旧 revision；再说明已经发生的活动应指向模板本身还是当时的 revision。

4. 事务、幂等、乐观锁与数据库约束

4.1 一个写命令的一生

`FinanceService.execute`在单个 `session.begin()` 中：领取来源回执、识别重放/冲突、执行 worker、写结果与审计，最后统一提交。中途异常会让业务行、审计和回执一起回滚。

来源幂等使用 `source_system + HMAC(source_event_id)` 的唯一范围和载荷指纹。原始来源 ID 不直接入库。`IdempotencyKeys` 和 canonical JSON 在 `service.py` 第 87 ~ 101 行，claim 逻辑从第 178 行开始。PostgreSQL 分支用 `INSERT ... ON CONFLICT DO NOTHING RETURNING id` 判断是否成功领取；同键同载荷重放原结果，同键异载荷返回 `duplicate_request_conflict`。

4.2 乐观版本与行锁

`Versioned.version_id` 在 `models.py` 第 41 ~ 42 行。调用方提交 `expected_version`；若对象已被别人改过，服务返回 `concurrent_modification`。退款、收入匹配和预算发布还会在 PostgreSQL 上用 `SELECT ... FOR UPDATE` 锁定关键行，例如退款路径见 `service.py` 第 507 ~ 513 行。

乐观版本回答“我看到的还是 v1 吗”，行锁回答“提交期间谁先操作这行”。二者和唯一/CHECK/FK 约束互补。

4.3 数据库约束是最后防线

应用校验让错误更友好，数据库约束防止绕过 service 或并发竞态造成坏数据。不能只选一个。月度快照的只读查询从 `service.py` 第 926 行开始；在 PostgreSQL 上要验证合适的事务隔离，不能用 SQLite 结果代替。

小练习

模拟“服务已提交但客户端没收到响应”。解释为什么重用同一来源键会返回旧结果，而换一个新键可能造成第二次业务效果。

5. SQLite 与 PostgreSQL：开发速度和生产语义分工

SQLite 用于快速本地迁移、绝大多数服务规则、回滚和查询测试；PostgreSQL 是目标环境，用于验证真实类型、锁、并发、隔离、触发器、标识符和 `timestampz`。

必须在 PostgreSQL 复验的行为包括：

- 两连接同时领取相同来源键；
- 并发退款、转账和预算发布；
- 延迟平衡触发器在提交时拒绝不平账交易；
- `SELECT FOR UPDATE` 与乐观版本配合；
- 只读 `REPEATABLE READ` 月度快照；
- `timestampz` aware UTC 往返；
- 空 schema 真实迁移、约束名和方言差异。

缺陷带来的四个教训

缺陷	根因	工程教训
C4-DATA-001	每次小额退款独立分余数，累计比例漂移	财务分配要验证累计结果，不能只测单次总额
C4-DATA-002	SQLite 动态类型允许文本进入 BIGINT affinity	本地库需额外 <code>typeof(...)= 'integer'</code> 防线
C4-DATA-003	SQLite 读回 timezone 列得到 naive datetime	公开边界统一恢复 aware UTC，并测月界
PG-C7-DATA-001	psycpg 下 <code>rowcount</code> 不能可靠判断 claim	使用 <code>RETURNING</code> 的实际返回值，不把 SQLite/驱动假设外推

P1 R2 本地证据为独立 70 通过、执行方财务 28 通过；后续真实 PostgreSQL 定向复验中 P1 8 项、P2 4 项和相邻 SQLite 9 项通过，claim 缺陷关闭。证据来源：[phase-1-c4-data-report.md](#)与[phase-2-c7-r2-postgresql-report.md](#)。这些数字对应固定测试范围，不是“数据库永远正确”的证明。

常见误区

1. 认为 ORM 模型存在就等于数据库已迁移。
2. 用 float 保存人民币，期待多次加总永远精确。
3. 在 repository 内随意 `commit()`，破坏 service 的原子事务。
4. 只做应用校验，不建数据库约束。
5. 把 SQLite 单写者行为当成 PostgreSQL 行锁证据。
6. 通过物理删除“修正”账本历史。

阶段练习

用虚拟数据设计一笔 30 元支出、10 元部分退款。列出两次交易头、每次分录、关联原交易、回执和预期余额。然后指出哪几步必须在同一事务内。

检查题

1. 交易头/分录模型比简单收支表多解决了哪些业务？
2. DTO 与 ORM 模型为何不能互相替代？
3. 同键同载荷与同键异载荷分别怎样处理？
4. 乐观版本和 PostgreSQL 行锁各解决什么竞争？
5. 为什么真实 PostgreSQL 迁移是发布门禁而非可选检查？